

Python for Everybody

Course 4: Using Databases with Python

A Comprehensive Study Guide

Modules Covered

Module 1 — Object-Oriented Python

Module 2 — Basic Structured Query Language (SQL)

Module 3 — Data Model and Relational SQL

Module 4 — Many-to-Many Relationships

Module 5 — Databases and Visualisation

Module 1: Object-Oriented Python

Object-Oriented Programming (OOP) is a programming paradigm that organises software design around data, or objects, rather than functions and logic. Python is inherently an object-oriented language — everything in Python, from integers to functions, is an object. Understanding OOP is fundamental to writing clean, maintainable, and scalable production-ready data science and AI applications.

SECTION 1 — THEORY

1.1 The Four Pillars of Object-Oriented Programming

OOP is built upon four foundational principles, commonly referred to as the Four Pillars. Each pillar serves a specific purpose in ensuring that software systems are modular, extensible, and easy to maintain.

1.1.1 Encapsulation

Encapsulation is the practice of bundling data (attributes/variables) and the methods (functions) that operate on that data into a single unit called a class. In addition to grouping related behaviour, encapsulation enforces data hiding — the internal state of an object is protected from direct, uncontrolled external access.

In Python, data hiding is achieved through name mangling: prefixing an attribute name with two underscores (e.g., `__balance`) instructs Python to internally rename the attribute to `__ClassName__balance`, effectively making it inaccessible from outside the class through its original name. Access to such private attributes is instead provided through controlled getter and setter methods.

Key Insight

Encapsulation separates the interface of a class (what it does) from its implementation (how it does it).

This is critical in production systems: you can change the internal logic of a class without breaking external code that uses it.

1.1.2 Abstraction

Abstraction is the principle of exposing only the essential features of an object while hiding the underlying complexity. It allows a developer to define a contract — what an object must do — without specifying how it must do it.

In Python, abstraction is formally implemented using the `abc` module (Abstract Base Classes). A class that inherits from ABC can declare abstract methods using the `@abstractmethod` decorator. Any concrete subclass that inherits from an abstract class is obligated to provide its own implementation of every abstract method. Attempting to instantiate an abstract class directly raises a `TypeError`, enforcing the contract.

Key Insight

Abstraction and Polymorphism are closely related.

When multiple subclasses implement the same abstract method differently, invoking that method on different objects produces different results — this is Polymorphism in action.

1.1.3 Inheritance

Inheritance is a mechanism by which a child class (also called a derived or subclass) acquires the attributes and methods of a parent class (also called a base or superclass). This promotes code reusability and establishes a natural hierarchical relationship between classes.

Python supports single inheritance (one parent), multiple inheritance (multiple parents), and multilevel inheritance (a chain of parent-child relationships). When a child class is instantiated, it has access to all public and protected members of its parent. The child class can also override inherited methods to provide specialised behaviour.

Key Insight

Inheritance implements the 'IS-A' relationship. For example, a Car IS-A Vehicle.

Always use inheritance judiciously. Overusing it leads to tightly coupled code. Favour composition over inheritance when there is no clear IS-A relationship.

1.1.4 Polymorphism

Polymorphism, derived from the Greek words for 'many forms', allows the same method name or interface to behave differently depending on the object that invokes it. It enables writing generic, flexible code that can operate on objects of different types without knowing the exact type at compile time.

In Python, polymorphism is most commonly achieved through method overriding (a subclass provides a new implementation of a method defined in a parent class) and duck typing (if an object implements the required method, it can be used regardless of its class hierarchy). Note: Python does not support true method overloading (same method name, different signatures) as seen in statically typed languages, but it can be simulated using default arguments or *args.

Term	Definition
Method Overriding	A subclass redefines a method inherited from its parent class, replacing its behaviour.
Method Overloading	Defining the same method with different parameter signatures. Python achieves this via default arguments.
Duck Typing	If an object walks like a duck and quacks like a duck, Python treats it as a duck — regardless of its actual class.

1.2 Summary of OOP Pillars

Term	Definition
Encapsulation	Bundles data and methods; restricts direct access to internal state via private attributes.

Abstraction	Hides implementation complexity; exposes only essential interfaces via abstract classes and methods.
Inheritance	Enables a child class to reuse and extend the functionality of a parent class.
Polymorphism	Allows the same method to behave differently across different object types.

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

The following cells constitute a complete, self-contained IPython notebook demonstrating all four OOP pillars in Python. Each cell is accompanied by a detailed explanation of every statement and its purpose.

Notebook: Object-Oriented Python in Practice

Cell 1: Encapsulation — BankAccount Class

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          # Public attribute
        self.__balance = balance    # Private attribute (name mangling)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f'Deposited {amount}. New balance: {self.__balance}')
        else:
            print('Deposit amount must be positive.')

    def get_balance(self):           # Controlled getter method
        return self.__balance

# --- Usage ---
account = BankAccount('Alice', 1000)
account.deposit(500)
print(account.get_balance())        # Output: 1500
# print(account.__balance)          # AttributeError: private attribute
```

Explanation

The `BankAccount` class is defined with two attributes: `owner` (public) and `__balance` (private). The double-underscore prefix on `__balance` triggers Python's name mangling mechanism, internally renaming it to `_BankAccount__balance`. This prevents accidental modification from outside the class.

The `__init__` method is the constructor — it is automatically called when a new `BankAccount` object is instantiated. It accepts `owner` and `balance` as arguments and assigns them to instance attributes.

The `deposit` method first validates that the amount is positive before modifying `__balance`. This is a critical aspect of encapsulation: external code cannot directly alter the balance to an invalid state; it must go through this controlled method.

The `get_balance` method serves as the getter — it provides read-only access to the private `__balance` attribute.

The commented-out line at the bottom demonstrates that direct access to `account.__balance` from outside the class raises an `AttributeError`, confirming that the private attribute is properly protected.

Cell 2: Abstraction — Abstract Vehicle Class

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print('Car engine started: turning ignition key.')

class Bike(Vehicle):
    def start(self):
        print('Bike engine started: pressing electric start button.')

# --- Usage ---
v1 = Car()
v2 = Bike()
v1.start()    # Output: Car engine started: turning ignition key.
v2.start()    # Output: Bike engine started: pressing electric start button.

# Attempting to instantiate the abstract class raises TypeError:
# v3 = Vehicle()    # TypeError: Can't instantiate abstract class Vehicle
```

Explanation

The `abc` module (Abstract Base Classes) is imported. `ABC` is the base class from which all abstract classes must inherit. The `abstractmethod` decorator is used to declare abstract methods.

The `Vehicle` class inherits from `ABC`, making it an abstract class. It declares a single abstract method, `start`. The `pass` keyword inside `start` simply satisfies Python's syntax requirement — the method body is intentionally empty because the abstract class does not define behaviour; it only mandates a contract.

The `Car` and `Bike` classes each inherit from `Vehicle`. Because `Vehicle` has an abstract method (`start`), both `Car` and `Bike` are required to provide their own concrete implementations of `start`. If either subclass fails to implement `start`, Python will raise a `TypeError` when you attempt to instantiate that subclass.

The final commented-out line illustrates that attempting to instantiate `Vehicle` directly raises a `TypeError`, which is the intended enforcement mechanism of abstraction.

Note how `v1.start()` and `v2.start()` — the same method call — produce different outputs. This is polymorphism emerging naturally from abstraction.

Cell 3: Inheritance — Parent and Child Classes

```
class Animal:
```

```
def __init__(self, name):
    self.name = name

def greet(self):
    print(f'Hello, I am {self.name}, an Animal.')

class Dog(Animal):
    # Dog inherits from Animal
    def __init__(self, name, breed):
        super().__init__(name) # Call parent constructor
        self.breed = breed

    def greet(self):
        # Method overriding
        print(f'Woof! I am {self.name}, a {self.breed}.')

class Cat(Animal):
    # Cat inherits from Animal
    pass # No additional attributes or methods

# --- Usage ---
generic_animal = Animal('Beast')
dog = Dog('Rex', 'Labrador')
cat = Cat('Whiskers')

generic_animal.greet() # Hello, I am Beast, an Animal.
dog.greet()           # Woof! I am Rex, a Labrador.
cat.greet()           # Hello, I am Whiskers, an Animal. (inherited)
```

Explanation

The Animal class serves as the parent (base) class. It defines a constructor (`__init__`) that accepts name and a method `greet` that prints a generic greeting.

The Dog class inherits from Animal (specified in parentheses: `class Dog(Animal)`). Its constructor first calls `super().__init__(name)` — the `super()` function returns a proxy object that represents the parent class, and calling `__init__` on it initialises the name attribute as defined in Animal. Dog then adds its own attribute `breed`.

Dog also overrides the `greet` method. When `dog.greet()` is called, Python uses the Dog version of `greet`, not Animal's. This is method overriding.

The Cat class simply inherits from Animal with `pass`, meaning it acquires all of Animal's attributes and methods unchanged. When `cat.greet()` is called, since Cat has no `greet` of its own, Python travels up the inheritance chain (Method Resolution Order, or MRO) and finds `greet` in Animal.

The `super()` function is the preferred and most Pythonic way to call the parent class's methods, as it correctly handles multiple inheritance scenarios.

Cell 4: Polymorphism — Duck Typing and Method Overriding

```
class Bird:
    def sound(self):
        return 'Tweet'

class Dog:
    def sound(self):
        return 'Bark'

class Cat:
    def sound(self):
```

```
        return 'Meow'

def make_sound(animal):
    # This function works with ANY object that has a .sound() method
    print(f'{animal.__class__.__name__} says: {animal.sound()}')

# --- Usage ---
animals = [Bird(), Dog(), Cat()]

for creature in animals:
    make_sound(creature)

# Output:
# Bird says: Tweet
# Dog says: Bark
# Cat says: Meow
```

Explanation

Bird, Dog, and Cat are three independent classes — none of them inherit from a common parent. Each implements a method called `sound` that returns a different string.

The `make_sound` function accepts any argument `animal` and calls `animal.sound()`. Python does not check the type of `animal` at the time the function is defined. As long as the object passed in has a `sound` method, the call will succeed. This is Python's duck typing in action: 'If it has a `sound` method, treat it as an animal that can make a sound.'

`animal.__class__.__name__` is an introspection expression: `__class__` returns the class of the object, and `__name__` returns the class name as a string. This provides clear, self-documenting output.

The `for` loop iterates over a list containing one instance of each class. Despite the objects being of completely different types, `make_sound` handles all of them uniformly. This demonstrates how polymorphism enables the writing of generic, reusable code.

This pattern is extensively used in data science frameworks such as `scikit-learn`, where all estimator objects expose a common `fit/predict` interface regardless of the underlying algorithm.

Module 2: Basic Structured Query Language (SQL)

A solid understanding of relational databases and SQL is indispensable for any data scientist or data engineer. Virtually all enterprise-grade data systems store structured data in relational databases, and SQL remains the primary language for querying, manipulating, and managing that data. This module establishes the theoretical and practical foundations required to work proficiently with relational databases.

SECTION 1 — THEORY

2.1 Relational Databases

A relational database is a structured collection of data that is organised into tables. Each table consists of rows (records/tuples) and columns (fields/attributes). The relational model, introduced by Edgar F. Codd in 1970, is founded on the principles of set theory and predicate logic, providing a mathematically rigorous framework for data management.

The key strength of a relational database lies in its ability to represent and query relationships between multiple tables efficiently. Rather than duplicating data, related information is stored in separate tables and linked via keys, ensuring data integrity and minimising redundancy.

Core Terminology

Term	Definition
Database	A collection of one or more related tables, views, and other database objects.
Table / Relation	A two-dimensional structure consisting of rows and columns, representing a specific entity (e.g., Customers, Products).
Tuple / Row	A single record within a table; represents one instance of the entity (e.g., one customer).
Attribute / Column	A named field within a table that holds a specific piece of data for each tuple (e.g., customer_name).
Primary Key	A column (or set of columns) whose values uniquely identify every row in a table. Cannot be NULL.
Foreign Key	A column in one table that references the Primary Key of another table, establishing a relationship.
Schema	The formal definition of a database's structure — its tables, columns, data types, and constraints.

2.2 Structured Query Language (SQL)

SQL (Structured Query Language) is the standardised declarative language used to communicate with relational database management systems (RDBMS). It is declarative in nature — the

developer specifies what data is needed rather than how to retrieve it; the DBMS's query optimiser determines the execution strategy.

SQL commands are categorised into five sub-languages:

Term	Definition
DDL — Data Definition Language	Commands that define and modify database structure: CREATE, ALTER, DROP, TRUNCATE.
DML — Data Manipulation Language	Commands that manage data within tables: INSERT, UPDATE, DELETE.
DQL — Data Query Language	Commands that retrieve data: SELECT (sometimes grouped under DML).
TCL — Transaction Control Language	Commands that manage database transactions: COMMIT, ROLLBACK, SAVEPOINT.
DCL — Data Control Language	Commands that control access permissions: GRANT, REVOKE.

2.3 Database Roles in Large Projects

In production environments, database-related responsibilities are typically shared between two complementary roles:

- **Application Developer:** Designs and builds the application logic and user interface. Works with the database primarily through SQL queries embedded in application code. Monitors the application for performance issues and logic errors.
- **Database Administrator (DBA):** Responsible for the design, implementation, maintenance, performance tuning, backup, and recovery of the database. Develops and enforces database policies, manages user access, and monitors query performance. Often collaborates with the Application Developer to co-design the data model.

2.4 Major Database Management Systems

Term	Definition
Oracle	Enterprise-scale, commercial RDBMS. Highly feature-rich and tunable; suited for mission-critical systems.
MySQL	Open-source, commercially supported RDBMS. Known for speed, reliability, and broad adoption in web applications.
SQL Server	Microsoft's RDBMS. Tightly integrated with the Windows ecosystem and Microsoft tooling (Power BI, SSRS, etc.).
SQLite	Serverless, file-based RDBMS. Embedded directly into applications. Ideal for local development, prototyping, and lightweight deployments.
PostgreSQL	Advanced open-source RDBMS with strong standards compliance, extensibility, and support for complex data types.

2.5 SQLite Browser

SQLite is the most widely deployed database engine in existence — it is embedded in every Android and iOS device, every modern web browser, and countless desktop applications. For learning and prototyping purposes, SQLite is the recommended choice because it requires no server setup: the entire database is stored in a single file.

DB Browser for SQLite (DB4S) is a free, open-source graphical tool that provides a visual interface to create, design, and edit SQLite database files. It supports the full spectrum of SQL command categories: DDL, DML, TCL, DCL, and DQL.

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

The following notebook cells demonstrate working with an SQLite database using Python's built-in `sqlite3` module.

Notebook: Working with SQLite in Python

Cell 1: Importing the `sqlite3` Module and Creating a Connection

```
import sqlite3

# Connect to an SQLite database.
# If the file does not exist, SQLite creates it automatically.
conn = sqlite3.connect('course4_demo.db')

# Create a cursor object to execute SQL commands
cursor = conn.cursor()

print('Connection established successfully.')
```

Explanation

`import sqlite3` loads Python's built-in `sqlite3` module, which provides a DB-API 2.0 compliant interface to SQLite databases. No third-party installation is required.

`sqlite3.connect('course4_demo.db')` establishes a connection to the SQLite database file named `course4_demo.db`. If this file does not exist in the current working directory, SQLite will create it automatically. The connection object (`conn`) represents the session with the database.

`conn.cursor()` creates a `Cursor` object. The cursor is the primary interface for executing SQL statements and fetching results. Think of it as a pointer or agent that interacts with the database on behalf of the connection.

Cell 2: Creating Tables using DDL (CREATE TABLE)

```
# DDL: Create a 'Users' table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS Users (
        user_id    INTEGER PRIMARY KEY AUTOINCREMENT,
        username   TEXT     NOT NULL UNIQUE,
        email      TEXT     NOT NULL,
```

```
        age        INTEGER
    )
'''

conn.commit()    # Persist changes to the database file
print('Table created successfully.')
```

Explanation

`cursor.execute()` sends a single SQL statement to the database engine for execution. Here it executes a CREATE TABLE statement.

CREATE TABLE IF NOT EXISTS ensures that the command only creates the table if it does not already exist, preventing errors on repeated execution — a best practice in production scripts.

The column definitions specify: `user_id` as an INTEGER that serves as the PRIMARY KEY and is auto-incremented (SQLite assigns a unique integer to each new row automatically); `username` as TEXT that is NOT NULL (mandatory) and UNIQUE (no two users can share the same username); `email` as mandatory TEXT; and `age` as an optional INTEGER.

`conn.commit()` is critical: SQLite (and most RDBMS) operate within transactions. The `commit()` call persists all changes made since the last commit to the physical database file. Without it, changes remain in a temporary transaction and will be lost if the connection is closed.

Cell 3: Inserting Data using DML (INSERT INTO)

```
# DML: Insert sample records
users_data = [
    ('alice_dev', 'alice@example.com', 28),
    ('bob_analyst', 'bob@example.com', 34),
    ('carol_ds', 'carol@example.com', 25),
]

cursor.executemany(
    'INSERT INTO Users (username, email, age) VALUES (?, ?, ?)',
    users_data
)

conn.commit()
print(f'{cursor.rowcount} rows inserted.')
```

Explanation

`users_data` is a list of tuples. Each tuple represents one complete row to be inserted into the Users table.

`cursor.executemany()` is an efficient method for executing the same SQL statement multiple times with different parameter values. It is significantly faster than calling `cursor.execute()` in a loop for bulk inserts.

The SQL statement uses positional placeholders (`?`). SQLite replaces each `?` with the corresponding value from the tuple at execution time. This approach — called parameterised queries — is the correct and mandatory way to pass user-supplied data to SQL statements. It prevents SQL injection attacks, which are a critical security vulnerability.

`cursor.rowcount` returns the number of rows affected by the last execute or executemany call, providing a useful confirmation of how many rows were actually inserted.

Cell 4: Querying Data using DQL (SELECT)

```
# DQL: Retrieve all users older than 26
cursor.execute('SELECT user_id, username, age FROM Users WHERE age > ?', (26,))

results = cursor.fetchall()    # Retrieve all matching rows

print(f'{ "ID":<5} { "Username":<15} { "Age":<5} ')
print('-' * 25)
for row in results:
    print(f'{row[0]:<5} {row[1]:<15} {row[2]:<5}')
```

Explanation

The SELECT statement retrieves user_id, username, and age from the Users table, filtered to only return rows where age is greater than 26. The parameterised placeholder (?) is used again for the filter value.

Note that the parameter is passed as a tuple (26,) — even for a single value. The trailing comma is essential because sqlite3 expects an iterable of parameters; without the comma, Python would interpret (26) as simply the integer 26, not a tuple, causing an error.

cursor.fetchall() retrieves all rows returned by the query and returns them as a list of tuples. Alternative methods include fetchone() (retrieves a single row) and fetchmany(n) (retrieves up to n rows), which are preferable when dealing with very large result sets to manage memory efficiently.

The formatted print statement uses Python f-strings with format specifiers (<5, <15) to produce aligned, tabular output — a simple but effective way to display query results in a notebook.

Cell 5: Closing the Connection

```
# Always close the connection when done
cursor.close()
conn.close()
print('Database connection closed.')
```

Explanation

cursor.close() releases the cursor object, freeing any resources it held.

conn.close() closes the database connection and releases all associated resources. Unclosed connections can lead to resource leaks and database locking issues, particularly in long-running applications.

In production code, it is best practice to manage database connections using Python's context manager protocol (the with statement), which guarantees that the connection is properly closed even if an exception occurs: with sqlite3.connect('db.db') as conn: ...

Module 3: Data Model and Relational SQL

Designing a sound database schema is one of the most consequential decisions in building a data-driven application. A poorly designed schema leads to data anomalies, redundancy, and maintenance nightmares, while a well-designed schema is the backbone of a robust, performant system. This module covers the principles of database design, entity-relationship modelling, normalisation, and the JOIN operation — the cornerstone of relational queries.

SECTION 1 — THEORY

3.1 Database Design Principles

Database design is a disciplined process of defining the structure, organisation, and relationships of data to meet both current and anticipated future requirements of an application. Good database design requires a balance of technical knowledge, domain understanding, and foresight.

- **Eliminate Redundancy:** The same piece of data should be stored in exactly one place. Redundant data wastes storage and, more critically, creates the risk of update anomalies — where updating data in one location leaves stale copies elsewhere.
- **Ensure Data Integrity:** Constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK) are the database's mechanism for enforcing business rules automatically, independent of application logic.
- **Design for Queries:** The schema should be designed with the anticipated query patterns in mind. Frequently joined columns should be indexed.
- **Use Surrogate Keys:** Prefer system-generated integer primary keys (AUTOINCREMENT/SERIAL) over natural keys (e.g., email addresses), as natural keys can change over time.

3.2 Entity-Relationship (ER) Diagrams

An Entity-Relationship (ER) diagram is a visual modelling tool used to represent the conceptual data model of a system before it is implemented in a database. It provides a language-agnostic blueprint that can be communicated to both technical and non-technical stakeholders.

An ER diagram consists of three primary components:

Term	Definition
Entity	Represents a real-world object or concept about which data is stored. Drawn as a rectangle. Example: Customer, Product, Order.
Attribute	A property or characteristic of an entity. Drawn as an oval. Example: Customer has attributes: customer_id, name, email.
Relationship	Describes the association between two entities. Drawn as a diamond. Example: Customer PLACES Order.

3.3 Building a Data Model — The Golden Rule

The single most important rule in relational database design is: Do not store the same string data in multiple places. Instead, store the data once and use a relationship (foreign key reference) to link to it.

Consider a music application: if you store the album name as a text string in every track record, you have thousands of copies of 'Abbey Road' scattered across the database. If the album title ever needs to be corrected, every single row must be updated. Instead, create a separate Album table, store the album title once, and have each Track record store only the `album_id` (foreign key) that references the Album table.

The Database Designer's Mantra

One thing in the real world = one record in the database.

References (foreign keys) are cheap; data duplication is expensive.

3.4 Keys in Relational Databases

Term	Definition
Primary Key	Uniquely identifies each row in a table. Must be unique and non-NULL. Typically an auto-incremented integer (surrogate key).
Foreign Key	A column in Table B that references the Primary Key of Table A. Enforces referential integrity — you cannot create a foreign key value that does not exist as a primary key in the referenced table.
Logical Key	A column that has real-world uniqueness meaning (e.g., email, ISBN, NationalID). Not necessarily the primary key, but often indexed for fast lookups.
Composite Key	A primary key composed of two or more columns. Used in junction tables for many-to-many relationships.

3.5 The JOIN Operation

The JOIN operation is the mechanism by which related data spread across multiple tables is combined into a single, unified result set. It is the most powerful and frequently used feature of relational databases.

Term	Definition
INNER JOIN	Returns only rows where there is a matching record in both tables. The most common join type.
LEFT JOIN	Returns all rows from the left table plus matching rows from the right table. Non-matching right rows appear as NULL.
RIGHT JOIN	Returns all rows from the right table plus matching rows from the left table. Non-matching left rows appear as NULL.
FULL OUTER JOIN	Returns all rows from both tables. Non-matching rows on either side appear as NULL. (Not supported in SQLite.)

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)**Notebook: Relational Schema Design and JOIN Queries****Cell 1: Setup — Creating a Relational Schema (Album and Track)**

```
import sqlite3

conn = sqlite3.connect(':memory:') # In-memory DB for quick demos
cursor = conn.cursor()

# Create the Album table (parent)
cursor.execute('''
    CREATE TABLE Album (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL UNIQUE
    )
''')

# Create the Track table (child) with FK to Album
cursor.execute('''
    CREATE TABLE Track (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        rating INTEGER CHECK(rating BETWEEN 1 AND 5),
        len INTEGER,
        count INTEGER DEFAULT 0,
        album_id INTEGER REFERENCES Album(id) ON DELETE CASCADE
    )
''')

conn.commit()
print('Schema created.')
```

Explanation

`sqlite3.connect(':memory:')` creates a transient, in-memory SQLite database. This is ideal for demonstrations and testing: it requires no file on disk and is destroyed when the connection is closed.

The Album table is the parent table. Its `id` column is the **PRIMARY KEY** with **AUTOINCREMENT**, and `title` is marked **UNIQUE** to enforce the golden rule — each album title is stored exactly once.

The Track table is the child table. It contains an `album_id` column that is a foreign key referencing `Album(id)`. The **REFERENCES** clause establishes the relationship. **ON DELETE CASCADE** means that if an Album record is deleted, all associated Track records are automatically deleted as well, maintaining referential integrity.

The `CHECK(rating BETWEEN 1 AND 5)` constraint is a database-level validation rule that ensures the `rating` column can only hold values from 1 to 5. This is superior to enforcing this rule only in application code, as it protects data integrity regardless of which application or script accesses the database.

Cell 2: Inserting Relational Data Correctly

```
# Insert albums first (parent records must exist before child records)
cursor.executemany(
    'INSERT INTO Album (title) VALUES (?)',
    [('Abbey Road',), ('Thriller',), ('Kind of Blue',)]
)

# Retrieve auto-generated IDs for albums
cursor.execute('SELECT id, title FROM Album')
albums = {title: aid for aid, title in cursor.fetchall()}
print('Album IDs:', albums)

# Insert tracks referencing correct album_ids
tracks = [
    ('Come Together', 5, 259, 10, albums['Abbey Road']),
    ('Something', 5, 182, 8, albums['Abbey Road']),
    ('Billie Jean', 5, 294, 15, albums['Thriller']),
    ('Flamenco Sketches', 5, 566, 5, albums['Kind of Blue']),
]
cursor.executemany(
    'INSERT INTO Track (title, rating, len, count, album_id) VALUES'
    '(?, ?, ?, ?, ?)',
    tracks
)
conn.commit()
print(f'{cursor.rowcount} tracks inserted.')
```

Explanation

Parent records (Album) must always be inserted before child records (Track). Attempting to insert a Track with an `album_id` that does not yet exist in Album would violate the foreign key constraint and raise an error.

After inserting the albums, their auto-generated integer IDs are retrieved using a `SELECT` statement. A dictionary comprehension `{title: aid for aid, title in cursor.fetchall()}` builds a convenient lookup mapping from album title to album ID.

The tracks list is constructed using these real album IDs, not hardcoded integers. This approach is robust: the code correctly references whatever integer IDs the database assigned, rather than assuming a specific sequence.

Note how album names are not stored in the Track table at all — only the integer `album_id` foreign key. This is the golden rule in practice.

Cell 3: JOIN Query — Combining Album and Track Data

```
# INNER JOIN: Retrieve track titles alongside their album names
cursor.execute('''
    SELECT
        Track.title    AS track_title,
        Album.title    AS album_title,
        Track.rating,
        Track.len
    FROM Track
    INNER JOIN Album ON Track.album_id = Album.id
    ORDER BY Album.title, Track.title
''')

rows = cursor.fetchall()
```

```
print(f'{"Track":<25} {"Album":<15} {"Rating":<8} {"Length(s)":<8}')
print('-' * 60)
for r in rows:
    print(f'{r[0]:<25} {r[1]:<15} {r[2]:<8} {r[3]:<8}')
```

Explanation

The INNER JOIN clause links the Track and Album tables on the condition `Track.album_id = Album.id`. For each row in Track, SQLite looks up the Album row whose id matches the track's `album_id` and combines them into a single output row.

Table-qualified column names (`Track.title`, `Album.title`) are used to avoid ambiguity — both tables have a column named `title`, so the table prefix is necessary to specify which one is intended.

The AS keyword assigns aliases (`track_title`, `album_title`) to the selected columns, making the result set's column names more descriptive and programmer-friendly.

ORDER BY `Album.title`, `Track.title` sorts the results alphabetically by album title first, then by track title within each album — producing a neatly organised output.

This single SQL query, through the JOIN operation, reconstructs the logical relationship between tracks and their albums without any data duplication in the stored tables.

Module 4: Many-to-Many Relationships

Many-to-many relationships are among the most common and important structural patterns encountered in real-world data modelling. Understanding how to represent and query them correctly is essential for building normalised, production-grade database schemas.

SECTION 1 — THEORY

4.1 Types of Database Relationships

Term	Definition
One-to-One (1:1)	Each record in Table A is associated with exactly one record in Table B, and vice versa. Example: Person ↔ Passport.
One-to-Many (1:N)	One record in Table A is associated with zero, one, or many records in Table B. Each record in Table B is associated with exactly one record in Table A. Example: Department → Employees.
Many-to-Many (M:N)	Multiple records in Table A are associated with multiple records in Table B, and vice versa. Example: Students ↔ Courses.

4.2 One-to-Many Relationships

A One-to-Many (1:N) relationship is modelled by placing a foreign key column in the 'many' side of the relationship. In a Department–Employee example, a single department employs many employees, but each employee belongs to exactly one department. Therefore, the foreign key (`dept_id`) resides in the Employee table, pointing back to the Department table.

The perspective determines the naming: from Department to Employee, it is One-to-Many (one department has many employees); from Employee to Department, it is Many-to-One (many employees belong to one department).

4.3 Many-to-Many Relationships

A Many-to-Many (M:N) relationship cannot be represented directly between two tables using simple foreign keys. Consider Students and Courses: a student enrolls in many courses, and a course has many enrolled students. Placing a single foreign key in either table would be insufficient and would create severe data integrity problems.

The standard solution is to introduce a junction table (also called a bridge table, associative table, or linking table). The junction table contains at minimum two foreign keys — one referencing each of the two related tables — and its primary key is typically the composite of these two foreign keys.

Design Rule for Many-to-Many

When you identify a Many-to-Many relationship, always decompose it into two One-to-Many relationships via a junction table.
The junction table may also carry additional attributes that describe the relationship itself (e.g., enrolment_date, grade).

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

Notebook: Many-to-Many — Students and Courses

Cell 1: Schema Creation — Students, Courses, and Junction Table

```
import sqlite3

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Parent table 1: Students
cursor.execute('''
    CREATE TABLE Students (
        student_id INTEGER PRIMARY KEY,
        student_name TEXT NOT NULL
    )
''')

# Parent table 2: Courses
cursor.execute('''
    CREATE TABLE Courses (
        course_id INTEGER PRIMARY KEY,
        course_name TEXT NOT NULL
    )
''')

# Junction table: resolves the M:N relationship
cursor.execute('''
    CREATE TABLE Student_Courses (
        student_id INTEGER,
        course_id INTEGER,
        enrolment_date TEXT,
        PRIMARY KEY (student_id, course_id),
        FOREIGN KEY (student_id) REFERENCES Students(student_id),
        FOREIGN KEY (course_id) REFERENCES Courses(course_id)
    )
''')

conn.commit()
print('M:N schema created.')
```

Explanation

Two independent parent tables are created: Students and Courses. Each has its own integer primary key. There is no direct relationship between them at this stage.

The Student_Courses junction table is the mechanism that implements the Many-to-Many relationship. It contains two foreign key columns: student_id (referencing Students) and course_id (referencing Courses).

PRIMARY KEY (student_id, course_id) defines a composite primary key. This means the combination of student_id and course_id must be unique, which naturally enforces the constraint that a student cannot be enrolled in the same course more than once.

enrolment_date is an additional attribute on the relationship itself — data that describes the association between a specific student and a specific course. This is a common pattern: junction tables often carry metadata about the relationship.

Cell 2: Inserting Data into All Three Tables

```
# Insert students
cursor.executemany('INSERT INTO Students VALUES (?, ?)', [
    (1, 'Alice'), (2, 'Bob'), (3, 'Carol')
])

# Insert courses
cursor.executemany('INSERT INTO Courses VALUES (?, ?)', [
    (101, 'Python Programming'),
    (102, 'Machine Learning'),
    (103, 'Database Systems'),
])

# Insert enrolments (many-to-many associations)
cursor.executemany(
    'INSERT INTO Student_Courses VALUES (?, ?, ?)',
    [
        (1, 101, '2024-09-01'), # Alice -> Python
        (1, 102, '2024-09-01'), # Alice -> ML
        (2, 101, '2024-09-02'), # Bob -> Python
        (2, 103, '2024-09-02'), # Bob -> Databases
        (3, 102, '2024-09-03'), # Carol -> ML
        (3, 103, '2024-09-03'), # Carol -> Databases
    ]
)
conn.commit()
print('Data inserted.')
```

Explanation

Students and Courses are inserted as independent entities with no reference to each other. Alice, Bob, and Carol are students; Python Programming, Machine Learning, and Database Systems are courses.

The enrolments are inserted into the junction table Student_Courses. Observe that Alice appears in two rows (enrolled in Python and ML), and Python appears in two rows (Alice and Bob are enrolled). This is the Many-to-Many relationship in action — one student relates to many courses, and one course relates to many students.

The data is stored compactly and without redundancy: the strings 'Alice' and 'Python Programming' are each stored only once, in their respective parent tables. The junction table stores only integer IDs.

Cell 3: Querying a Many-to-Many Relationship with Multiple JOINS

```
# Find all courses each student is enrolled in
cursor.execute('''
SELECT
```

```
s.student_name,  
c.course_name,  
sc.enrolment_date  
FROM Student_Courses AS sc  
INNER JOIN Students AS s ON sc.student_id = s.student_id  
INNER JOIN Courses AS c ON sc.course_id = c.course_id  
ORDER BY s.student_name, c.course_name  
''')  
  
rows = cursor.fetchall()  
for row in rows:  
    print(f'{row[0]:<10} enrolled in {row[1]:<25} on {row[2]}')
```

Explanation

Querying a Many-to-Many relationship requires two JOIN operations: the first joins Student_Courses with Students (to resolve student names from student IDs), and the second joins Student_Courses with Courses (to resolve course names from course IDs).

Table aliases (sc, s, c) are introduced using the AS keyword to make the query more concise and readable. Without aliases, every column reference would need to be fully qualified with the full table name.

The query correctly reconstructs the full picture of enrolments by navigating through the junction table — from student IDs to student names, and from course IDs to course names — all in a single SQL statement.

ORDER BY s.student_name, c.course_name ensures the output is sorted first by student name and then by course name within each student, producing an organised, readable result.

Module 5: Databases and Visualisation — Data Mining Foundations

The culmination of database proficiency in a data science context is the ability to extract meaningful insights from stored data and communicate those insights visually. This module introduces data mining — the discipline that bridges raw databases and actionable knowledge — and demonstrates end-to-end pipelines from SQL query to visual output using Python.

SECTION 1 — THEORY

5.1 What is Data Mining?

Data mining is the computational process of discovering previously unknown, non-trivial, and potentially useful patterns, relationships, trends, and anomalies within large datasets. It draws upon a multidisciplinary toolkit comprising statistics, machine learning, database systems, and information theory.

The key differentiator of data mining from simple querying is its focus on discovery: while a SQL `SELECT` statement retrieves data you already know exists, data mining algorithms reveal patterns you did not know were there. Examples include discovering that customers who purchase product A also tend to purchase product B (association rule mining), or identifying unusual spikes in transaction volume that may indicate fraud (anomaly detection).

5.2 The Data Mining Process — CRISP-DM

The Cross-Industry Standard Process for Data Mining (CRISP-DM) is the most widely adopted framework for structuring data mining and data science projects. It consists of six iterative phases:

1. Business Understanding: Define the business objective and translate it into a data mining problem.
2. Data Understanding: Collect, explore, and assess the quality of the available data.
3. Data Preparation: Clean, transform, and engineer features from raw data.
4. Modelling: Select and apply appropriate data mining / machine learning algorithms.
5. Evaluation: Assess whether the model meets the business objective.
6. Deployment: Integrate the model into production systems and monitor its performance.

5.3 Common Data Mining Techniques

Term	Definition
Classification	Assigns data points to predefined categories (e.g., spam detection, disease diagnosis).
Clustering	Groups data points into clusters based on similarity without predefined categories (e.g., customer segmentation).

Association Rules	Discovers co-occurrence patterns between items (e.g., market basket analysis).
Regression	Predicts a continuous numerical value (e.g., house price prediction).
Anomaly Detection	Identifies data points that deviate significantly from expected patterns (e.g., fraud detection).
Text Mining / NLP	Extracts structured information from unstructured text data (e.g., sentiment analysis).

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

The following cells demonstrate a complete pipeline: populating a SQLite database with sales data, querying it with SQL aggregation, and visualising the results using matplotlib — a workflow representative of real-world data science practice.

Notebook: End-to-End Pipeline — Database to Visualisation

Cell 1: Setup and Data Population

```
import sqlite3
import random
from datetime import date, timedelta

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create a Sales table
cursor.execute('''
    CREATE TABLE Sales (
        sale_id    INTEGER PRIMARY KEY AUTOINCREMENT,
        product    TEXT     NOT NULL,
        category   TEXT     NOT NULL,
        amount     REAL     NOT NULL,
        sale_date  TEXT     NOT NULL
    )
''')

# Populate with synthetic data
products = [
    ('Laptop', 'Electronics'), ('Smartphone', 'Electronics'),
    ('Desk Chair', 'Furniture'), ('Bookshelf', 'Furniture'),
    ('Python Book', 'Books'), ('ML Handbook', 'Books'),
]

random.seed(42)
start = date(2024, 1, 1)
records = []
for _ in range(200):
    product, category = random.choice(products)
    amount = round(random.uniform(10, 1500), 2)
    sale_date = str(start + timedelta(days=random.randint(0, 364)))
    records.append((product, category, amount, sale_date))
```

```
cursor.executemany(
    'INSERT INTO Sales (product, category, amount, sale_date) VALUES (?, ?, ?, ?)',
    records
)
conn.commit()
print(f'{len(records)} sales records inserted.')
```

Explanation

The `sqlite3`, `random`, `date`, and `timedelta` modules are imported. `random` is used to generate realistic synthetic data; `date` and `timedelta` facilitate date arithmetic for generating random sale dates within the year 2024.

The `Sales` table models a simplified sales transaction record. `amount` is stored as `REAL` (SQLite's floating-point type) to accommodate decimal currency values. `sale_date` is stored as `TEXT` in ISO 8601 format (YYYY-MM-DD) — SQLite's recommended approach for date storage, which preserves lexicographic sort order.

`random.seed(42)` initialises the random number generator with a fixed seed, ensuring the generated data is reproducible — a critical practice in data science experiments and notebooks.

The loop generates 200 synthetic sales records. `random.choice(products)` selects a random product-category pair; `random.uniform(10, 1500)` generates a random sale amount; and `timedelta(days=random.randint(0, 364))` produces a random date within 2024.

`cursor.executemany()` inserts all 200 records in a single, efficient bulk operation.

Cell 2: SQL Aggregation — Revenue by Category

```
# SQL aggregation: total revenue per category
cursor.execute('''
    SELECT
        category,
        COUNT(*)           AS num_sales,
        ROUND(SUM(amount), 2) AS total_revenue,
        ROUND(AVG(amount), 2) AS avg_sale_value
    FROM Sales
    GROUP BY category
    ORDER BY total_revenue DESC
''')

agg_results = cursor.fetchall()

print(f'{'Category':<15} {'# Sales':<10} {'Total Revenue':>15} {'Avg Sale':>12}')
print('=' * 55)
for row in agg_results:
    print(f'{row[0]:<15} {row[1]:<10} {row[2]:>15.2f} {row[3]:>12.2f}')
```

Explanation

This SQL statement uses aggregate functions: `COUNT(*)` counts all rows within each group; `SUM(amount)` totals the revenue within each group; `AVG(amount)` computes the mean sale value. `ROUND(..., 2)` restricts the output to two decimal places.

`GROUP BY category` is the key clause: it partitions all rows in the `Sales` table into groups sharing the same category value, and applies the aggregate functions independently to each group.

ORDER BY total_revenue DESC sorts the result from highest to lowest revenue, making it immediately apparent which category contributes most to overall sales — a common business intelligence requirement.

The formatted print output uses Python's f-string alignment specifiers: < for left-alignment and > for right-alignment, producing a professional-looking tabular report within the notebook output.

Cell 3: Visualisation — Bar Chart of Revenue by Category

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Extract data from query results
categories = [row[0] for row in agg_results]
total_revenues = [row[2] for row in agg_results]

# Create bar chart
fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(categories, total_revenues,
               color=['#2E75B6', '#1F3864', '#70AD47'],
               edgecolor='white', linewidth=0.8, width=0.55)

# Annotate bars with revenue values
for bar in bars:
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 200,
        f'${bar.get_height():.0f}',
        ha='center', va='bottom', fontsize=10, fontweight='bold'
    )

ax.set_title('Total Revenue by Product Category (2024)',
             fontsize=14, fontweight='bold', pad=15)
ax.set_xlabel('Product Category', fontsize=12)
ax.set_ylabel('Total Revenue (USD)', fontsize=12)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(
    lambda x, _: f'${x:,.0f}'
))
ax.set_ylim(0, max(total_revenues) * 1.2)
ax.spines[['top', 'right']].set_visible(False)
ax.grid(axis='y', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('revenue_by_category.png', dpi=150)
plt.show()
print('Chart saved.')
```

Explanation

matplotlib.pyplot provides the high-level interface for creating static visualisations. matplotlib.ticker is imported to customise axis tick formatting, allowing revenue values to be displayed with dollar signs and commas.

Two lists are constructed from the query results via list comprehensions: categories contains the category name strings, and total_revenues contains the corresponding floating-point revenue totals.

`fig, ax = plt.subplots(figsize=(8, 5))` creates a Figure object and an Axes object. The Axes (`ax`) is the actual plot area. Using the object-oriented API (`ax.bar`, `ax.set_title`, etc.) is the recommended approach for matplotlib, as it provides precise control over every element of the figure.

`ax.bar()` renders the bar chart. Custom hexadecimal colours are applied to each bar. `edgecolor='white'` and `linewidth=0.8` add subtle white borders to the bars, improving visual clarity.

The for loop over bars annotates each bar with its monetary value. `bar.get_x()` and `bar.get_width()` compute the horizontal centre of each bar; `bar.get_height()` gives the y-position of the bar top, and the offset + 200 places the text slightly above the bar.

`mticker.FuncFormatter` applies a custom formatting function to the y-axis tick labels, converting raw numbers (e.g., 45000) into readable currency strings (e.g., \$45,000).

`ax.spines[['top', 'right']].set_visible(False)` removes the top and right border lines, following modern data visualisation best practices for clean, publication-ready charts.

`plt.savefig()` saves the chart as a PNG file at 150 DPI — sufficient resolution for reports and presentations.

End of Course 4: Using Databases with Python

This document has covered the complete arc of Python for Everybody, Course 4 — from the principles of Object-Oriented Programming that underpin modern Python development, through the foundations of relational databases and SQL, to the design of normalised schemas and the extraction of insights through data mining and visualisation. Each concept has been presented with both rigorous theoretical explanation and practical, annotated code examples suitable for production-ready application development.